

深圳大学实验报告

课程名称： 数据库原理及应用

实验项目名称： 复杂查询

学院： 数学科学学院

专业： 信息与计算科学

指导教师： 郑海虹

报告人： 陈冠宇 学号： 2024382026 班级： 01

实验时间： 2026.04.014~21

实验报告提交时间： 2026.04.23

教务处制

实验目的与要求:

掌握连接查询, 嵌套查询, 集合查询

方法、步骤:

1. 本实验在“学生管理”数据库环境下进行, 先检查 `student`、`course`、`score` 三表结构与样例数据是否完整, 再按题目顺序逐条编写并执行 SQL。
2. 对“姓名+课程编号”“姓名+课程名+成绩”等查询, 采用等值连接与多表连接(`student` $\bowtie$ `score` $\bowtie$ `course`) 实现。
3. 对“包含未选课同学”的查询, 采用左外连接 (`LEFT JOIN`) 保留学生表全部记录。
4. 对“选修某课程/同时选修两门/只选修其一”等条件, 采用子查询结合 `IN`、`EXISTS`、集合逻辑 (交集/差集思想) 实现。
5. 对“高于平均分”问题, 先用聚合函数 `AVG()` 求平均值, 再在外层查询中比较入学成绩。
6. 对“高于所有男生/高于任意女生”问题, 采用子查询配合 `ALL`、`ANY` (或 `MAX/MIN` 等价写法) 实现。
7. 对“课程与先修课名称”问题, 采用课程表自连接 (同表起别名) 查询课程及其先修课对应关系。
8. 对“两名同学共同选课且成绩比较”问题, 采用成绩表自连接并增加学号与成绩比较条件。
9. 对“至少与某同学同修一门课”问题, 采用 `EXISTS` 或 `IN` 查询与目标同学课程集合有交集的学生。
10. 对“选修全部课程/选修了某同学全部课程”问题, 采用“双重 `NOT EXISTS`”实现全称条件判断。
11. 对“班级男女人数及合计”问题, 采用条件聚合 `SUM(CASE WHEN ... THEN 1 END)` 或 `GROUP BY ... WITH ROLLUP` 实现分组统计与总计行。
12. 对“每门课良好及以下人数分段统计”问题, 按课程分组, 使用 `SUM(CASE WHEN score_grade>80 THEN 1 ELSE 0 END)` 等条件统计实现区间计数。
13. 每条 SQL 执行后核对结果字段与题意一致性, 并保留关键结果截图用于“实验过程及内容”部分。

实验过程及内容：

1. 查询所有选课学生的姓名和选修课程的课程编号

```
SELECT s.student_name, sc.course_id  
FROM student s
```

```
JOIN score sc ON s.student_id = sc.student_id;
```

2. 查询所有选课学生的姓名、课程名称和成绩

```
SELECT s.student_name, c.course_name, sc.score_grade  
FROM student s
```

```
JOIN score sc ON s.student_id = sc.student_id
```

```
JOIN course c ON sc.course_id = c.course_id;
```

3. 查询所有学生的姓名、课程名称、成绩（包括未选课同学）

```
SELECT s.student_name, c.course_name, sc.score_grade  
FROM student s
```

```
LEFT JOIN score sc ON s.student_id = sc.student_id
```

```
LEFT JOIN course c ON sc.course_id = c.course_id;
```

4. 查询所有选修了“管理学”课程的学生名单

```
SELECT DISTINCT s.student_name  
FROM student s
```

```
JOIN score sc ON s.student_id = sc.student_id
```

```
JOIN course c ON sc.course_id = c.course_id
```

```
WHERE c.course_name = '管理学';
```

5. 查询同时选修了“管理学”和“计算机文化基础”两门课程的学生名单

```
SELECT s.student_name
```

```
FROM student s
```

```
WHERE s.student_id IN (
```

```
    SELECT sc.student_id
```

```
    FROM score sc
```

```
    JOIN course c ON sc.course_id = c.course_id
```

```
    WHERE c.course_name = '管理学'
```

```
)
```

```
AND s.student_id IN (
```

```
    SELECT sc.student_id
```

```
    FROM score sc
```

```
    JOIN course c ON sc.course_id = c.course_id
```

```
    WHERE c.course_name = '计算机文化基础'
```

```
);
```

6. 查询选修了“管理学”但没选修“计算机文化基础”课程的学生名单

```
SELECT s.student_name
```

```
FROM student s
```

```
WHERE s.student_id IN (
```

```
    SELECT sc.student_id
```

```
    FROM score sc
```

```
    JOIN course c ON sc.course_id = c.course_id
```

```

WHERE c.course_name = '管理学'
)
AND s.student_id NOT IN (
    SELECT sc.student_id
    FROM score sc
    JOIN course c ON sc.course_id = c.course_id
    WHERE c.course_name = '计算机文化基础'
);
7. 查询所有入学成绩高于平均入学成绩的学生姓名及其入学成绩
SELECT student_name, student_entrancescore
FROM student
WHERE student_entrancescore > (
    SELECT AVG(student_entrancescore) FROM student
);
8. 查询所有党员及选修 04010101 课程的学生的学号
SELECT student_id
FROM student
WHERE student_partymember = 0x01
UNION
SELECT student_id
FROM score
WHERE course_id = '04010101';
9. 查询入学成绩高于所有男同学入学成绩的女生姓名
SELECT student_name
FROM student
WHERE student_sex = '女'
AND student_entrancescore > ALL (
    SELECT student_entrancescore
    FROM student
    WHERE student_sex = '男'
);
10. 查询入学成绩高于任意一名女同学入学成绩的男生姓名
SELECT student_name
FROM student
WHERE student_sex = '男'
AND student_entrancescore > ANY (
    SELECT student_entrancescore
    FROM student
    WHERE student_sex = '女'
);
11. 查询所有开课课程的课程名称及先修课名称（只显示有先修课）
SELECT c.course_name AS 课程名称, p.course_name AS 先修课名称
FROM course c
JOIN course p ON c.course_prerequisite = p.course_id;

```

12. 查询 2021094002 与 2021094001 都选修且前者成绩更高的课程编号

```
SELECT a.course_id
FROM score a
JOIN score b ON a.course_id = b.course_id
WHERE a.student_id = '2021094002'
      AND b.student_id = '2021094001'
      AND a.score_grade > b.score_grade;
```

13. 查询至少与 2021094001 同学选修了同一门课程的学生名单

```
SELECT DISTINCT s.student_name
FROM student s
JOIN score sc ON s.student_id = sc.student_id
WHERE sc.course_id IN (
    SELECT course_id
    FROM score
    WHERE student_id = '2021094001'
);
```

14. 查询选修了全部课程的学生名单

```
SELECT s.student_name
FROM student s
WHERE NOT EXISTS (
    SELECT 1
    FROM course c
    WHERE NOT EXISTS (
        SELECT 1
        FROM score sc
        WHERE sc.student_id = s.student_id
              AND sc.course_id = c.course_id
    )
);
```

15. 查询选修了“2021094003”同学全部课程的学生名单（不包括本人）

```
SELECT s.student_name
FROM student s
WHERE s.student_id <> '2021094003'
      AND NOT EXISTS (
        SELECT 1
        FROM score t
        WHERE t.student_id = '2021094003'
              AND NOT EXISTS (
                SELECT 1
                FROM score sc
                WHERE sc.student_id = s.student_id
                      AND sc.course_id = t.course_id
            )
    );
```

16. 查询国际贸易 081 班男生人数和女生人数及合计信息

```
SELECT c.class_name AS 班级,
       SUM(CASE WHEN s.student_sex = '男' THEN 1 ELSE 0 END) AS 男生人数,
       SUM(CASE WHEN s.student_sex = '女' THEN 1 ELSE 0 END) AS 女生人数,
       COUNT(*) AS 合计
FROM student s
JOIN class c ON s.class_id = c.class_id
WHERE c.class_name = '国际贸易 081'
GROUP BY c.class_name;
```

17. 分段统计：每门课程良好以上及以下人数（良好指成绩>80）

```
SELECT sc.course_id AS 课程编号,
       SUM(CASE WHEN sc.score_grade > 80 THEN 1 ELSE 0 END) AS 良好以上人数,
       SUM(CASE WHEN sc.score_grade <= 80 THEN 1 ELSE 0 END) AS 良好及以下人
数
FROM score sc
GROUP BY sc.course_id;
```

实验结论及心得：

本次实验围绕复杂查询完成了连接查询、嵌套查询和集合查询的综合练习。通过多表连接、外连接、自连接、IN/EXISTS、ANY/ALL、

分组统计等语句，实现了课程筛选、成绩比较、全称条件判断和分类汇总等需求，基本达到了实验目标。

实验中体会最深的是：同一题目可以有多种写法，但要根据题意选择最准确、可读性更好的方案；尤其是“选修全部课程”“至少满足

一门课程”等问题，使用双重 NOT EXISTS 或集合思想更稳定。整体上，我对 SQL

复杂查询的逻辑组织和结果校验能力有了明显提升。

指导教师批阅意见:

成绩评定:

指导教师签字：郑海虹
2026 年 04 月 27 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。